DEPI GRADUATION PROJECT

Automated Testing Framework

for a Web Application

Document Info	
Test Plan ID	TP-DEPI-001
Project Name	DEPI Automated Testing Framework
Prepared By	Team QC Engineers
Date	April 2025
Version	1.0
Status	Active

Testing Websites

Website	URL / Purpose
Automation Exercise	https://automationexercise.com — Manual, API & Automation Testing
OpenCart Demo	https://demo.opencart.com — Database (MySQL) Testing

Team Members

Name	Role
Hassan Yehia Hassan Aboalersahd	Team Leader / QC Engineer

Ahemd Mahmoud Ahmed	QC Engineer
Hatem Ashraf Abdel Tawab Abdel Hamid	QC Engineer
Mohamed Essam Ahmed Salem	QC Engineer
Ziad Mahmoud Ahmed Ali Mostafa	QC Engineer

1. Introduction

This Test Plan governs the design, development, and execution of an automated testing framework for a web application as part of the DEPI Graduation Project. The framework covers end-to-end validation across UI, API, Database, and Performance layers using industry-standard tools and Agile Scrum practices.

The project uses two target websites:

- Automation Exercise (https://automationexercise.com) — primary website for Manual, Automation, and API testing.
- OpenCart Demo (https://demo.opencart.com) — used for Database (MySQL) testing to validate data integrity at the backend.

The testing effort is organized across four sprints following the Agile Scrum methodology, with daily standups, sprint reviews, and retrospectives.

2. Objectives

- Verify the full functional coverage of Automation Exercise — including user management, product browsing, cart, checkout, and API endpoints.
- Validate database integrity on OpenCart using MySQL queries (INSERT, SELECT, UPDATE, DELETE, JOINs).
- Build a scalable, maintainable automation framework using the Page Object Model (POM) pattern.
- Implement API test collections in Postman and automate them using Newman.
- Apply BDD (Cucumber + Gherkin) to map plain-English scenarios to automation scripts.
- Detect, log, and track all defects through Jira with proper severity and priority classification.
- Integrate the framework with Jenkins CI/CD for continuous automated testing.
- Validate non-functional requirements (load & stress) using Apache JMeter.
- Produce comprehensive test reports at each sprint milestone.

3. Scope

3.1 In Scope

Testing Type	Covered Features / Modules

Manual Testing (UI)	User Registration, Login, Logout, Delete Account, Product Browsing, Search, Add to Cart, Checkout, Contact Form, Subscription
API Testing (Postman)	All 26+ API endpoints: GET/POST/PUT/DELETE for Products, Brands, Users, Search, Cart
Database Testing (MySQL)	OpenCart DB: User data, product records, order tables, data integrity after CRUD operations
Automation Testing (Selenium + POM)	All core UI flows automated with reusable Page Object classes in Java
BDD (Cucumber)	Key scenarios written in Gherkin (.feature files) mapped to step definitions
Performance Testing (JMeter)	Load and stress tests on Automation Exercise key pages
CI/CD (Jenkins)	Automated pipeline triggering test suite on each commit via GitHub
Defect Management (Jira)	Bug logging, sprint tracking, test case management throughout all sprints

3.2 Out of Scope

- Security / Penetration testing
- Payment gateway integration testing (demo environment)
- Mobile native app testing
- Third-party SSO / OAuth flows
- Accessibility (WCAG) compliance testing

4. Test Items / Features to Be Tested

4.1 Automation Exercise — UI & API

Module	Features to Test
User Registration	Valid registration, duplicate email, blank fields, password mismatch
Login / Logout	Valid/invalid credentials, session handling, redirect on logout
Delete Account	Account deletion confirmation, session termination
Product Browsing	Category navigation (Women/Men/Kids), product detail page, image display
Search	Keyword search, partial match, no results scenario
Cart Management	Add to cart, update quantity, remove item, empty cart
Checkout Flow	Address form, payment form, order confirmation, order history

Contact Us Form	Valid submission, blank fields, file upload
Subscription	Email subscription (footer), confirmation message
API Endpoints	All 26+ listed endpoints: response codes, payload validation, error handling
Test Cases Page	Verify all 26+ test scenarios listed on the site execute correctly

4.2 OpenCart — Database Testing

DB Test Area	SQL Operations
User Data	SELECT registered users, verify INSERT on signup, UPDATE profile fields
Product Records	SELECT product inventory, verify price/stock data integrity
Order Tables	JOIN orders + order_items + customers, verify totals and status
Data Integrity	Verify cascading deletes, foreign key constraints, NULL handling

5. Test Approach

5.1 Testing Types

Testing Type	Tools	Details
Manual / Functional Testing	Jira, Browser	Positive & negative scenarios, exploratory sessions
UI Automation	Selenium WebDriver + Java	POM framework, TestNG runner
API Testing	Postman / Newman	Collections per endpoint group, assertions on status & payload
Database Testing	MySQL Workbench / JDBC	SQL queries to validate data after UI operations
BDD	Cucumber + Gherkin	Feature files for core user journeys
Performance Testing	Apache JMeter	Load (50 users), stress (200 users) on key pages
CI/CD	Jenkins + GitHub	Pipeline runs full suite on each PR/commit

5.2 Techniques

- Black-box testing (no source code access)
- Equivalence Partitioning — group inputs into valid/invalid classes
- Boundary Value Analysis — test min, max, just-inside, just-outside values for all form fields
- State Transition Testing — login states, cart states, order status transitions
- Exploratory Testing — unscripted sessions at the start of each sprint to discover gaps
- Decision Table Testing — checkout flow with multiple payment/address combinations

5.3 Test Levels

Level	Scope	Responsible
Unit	Individual Java helper/utility classes	Dev / Automation QC
Integration	API endpoint + DB consistency checks	QC Engineers
System	End-to-end UI + API + DB combined flows	Full Team
Acceptance	26+ documented test scenarios on Automation Exercise	Team Leader

6. Test Environment

Item	Details
Primary Test URL	https://automationexercise.com
DB Test URL	https://demo.opencart.com
Browsers	Google Chrome (latest), Brave (latest), Microsoft Edge (latest)
Operating System	Windows 11
IDE	IntelliJ IDEA / Eclipse
Automation Language	Java 17+
Test Framework	Selenium WebDriver 4.x, TestNG, Cucumber
API Tool	Postman (manual) + Newman (CI runner)
DB Tool	MySQL Workbench 8.x
Performance Tool	Apache JMeter 5.x
CI/CD	Jenkins (local or cloud)
Version Control	Git + GitHub
Bug Tracking	Jira (Scrum board)

7. Test Data

7.1 User Registration Test Data

Field	Valid Input	Invalid Input
Name	Ahmed Test	(blank)
Email	ahmed.test@example.com	invalid-email / already registered
Password	Test@1234	test (too simple)
Date of Birth	01/01/1995	future date / 00/00/0000
First Name	Ahmed	(blank)
Last Name	Mohamed	12345 (numeric)
Company	DEPI Corp	(optional — leave blank)
Address 1	123 Test Street	(blank)
City	Cairo	(blank)

State	Egypt	(blank)
Zip Code	12345	abcde / (blank)
Mobile Number	01012345678	abc / (blank)

7.2 API Test Data

- Product ID: valid IDs (1–34), invalid (9999), string ('abc')
- Brand ID: valid (1–34), invalid, empty
- Search term: 'top', 'jeans', empty string, special characters (@#$)
- User credentials: valid registered user, wrong password, non-existent email
- POST body payloads: complete valid JSON, missing required fields, wrong data types

7.3 Database Test Data (OpenCart)

- New user registration → verify INSERT in oc_customer table
- Product price update via admin → verify UPDATE in oc_product table
- Order placement → verify JOIN between oc_order + oc_order_product + oc_customer
- Account deletion → verify cascading DELETE from customer-related tables

8. Entry and Exit Criteria

8.1 Entry Criteria

- Test environment is accessible (both websites live)
- Jira project board configured with Sprint structure
- GitHub repository created and team members added
- Java + Selenium + TestNG + Cucumber dependencies set up in Maven/Gradle
- MySQL Workbench connected to OpenCart database
- Postman collections structure created
- Test Plan document reviewed and approved

8.2 Exit Criteria

- 100% of planned test cases executed
- All Critical and High severity defects resolved or deferred with justification
- Test coverage ≥ 90% of in-scope features
- Automation suite runs end-to-end with pass rate ≥ 85%
- Newman API test report generated with 0 critical failures
- JMeter performance report generated (response time < 3s under load)
- Jenkins CI/CD pipeline successfully triggers and passes
- Test Completion Report prepared and signed off

9. Sprint Breakdown (Agile Scrum)

Sprint 1 — Planning

Objective: Establish project foundation, scope, and test strategy.

Task	Deliverable
Choose graduation project websites	Project scope defined (Automation Exercise + OpenCart)
Define test schedule (timelines per sprint)	Sprint calendar and roadmap
Define Entry & Exit Criteria	Documented in Test Plan Section 8
Set up Jira Scrum board	Jira project with backlog, sprint columns
Create GitHub repository	Repo with README, folder structure, .gitignore
Team role assignment	Team charter document

Sprint 2 — Analysis & Design

Objective: Identify all test scenarios, design the framework skeleton, and write initial BDD feature files.

Task	Deliverable
Exploratory Testing sessions on Automation Exercise	Exploration notes + automation candidate list
UI Test Case Design (manual)	Test cases for all 26+ scenarios in Jira
API Test Case Design	Postman collection structure with test assertions
Database Test Case Design (OpenCart)	SQL query scripts for data validation
Automation Test Case Selection	List of POM-suitable test cases
BDD Feature Files (Gherkin)	First .feature files (Login, Registration, Search)
Initial automation framework setup	Maven project with POM classes for homepage + login

Sprint 3 — Implementation

Objective: Build working framework, configure all tools, and integrate version control.

Task	Deliverable
Git/GitHub — branching, PRs, code reviews	Active repo with feature branches + merged PRs
Full Postman API collection + Newman scripts	API test results (JSON/HTML report)
JMeter performance test scripts	Load/stress test plans for key pages
MySQL queries for OpenCart DB validation	SQL scripts file in repo
Expand Selenium POM automation scripts	Page classes for Cart, Checkout, Product pages
Map BDD step definitions to automation	Glue code connecting Gherkin → Selenium

Sprint 4 — Test Completion & Execution

Objective: Execute full test suite, optimize framework, deploy CI/CD, and produce final reports.

Task	Deliverable
Optimize framework (POM refactor, utilities, data-driven)	Clean, reusable framework code

Execute full UI automation suite	TestNG execution report
Execute full API test suite via Newman	Newman HTML report
Execute Database validation queries	DB test results document
Execute JMeter performance tests	JMeter dashboard report
Deploy Jenkins CI/CD pipeline	Jenkins job configuration + pipeline screenshot
Analyze results and log remaining defects in Jira	Defect log (Jira export)
Write full user guide documentation	framework-guide.md in GitHub
Prepare Test Completion Report	Final TCR document

10. Agile Practices

10.1 Daily Standup (15 min max)

- What did I complete yesterday?
- What will I work on today?
- What blockers am I facing?

10.2 Sprint Retrospective (End of Each Sprint)

- What went well this sprint?
- What did not go well?
- What will we improve in the next sprint?

Benefits: Transparency, continuous improvement, team alignment, and early blocker detection.

11. Tools Summary

Tool	Purpose
Selenium WebDriver (Java)	UI test automation
TestNG	Test runner, assertions, parallel execution
Cucumber (BDD)	Gherkin feature files + step definitions
Postman	Manual API testing, test scripts, collections
Newman	CLI runner for Postman collections (CI integration)

Apache JMeter	Load and stress performance testing
MySQL Workbench	Database query execution and validation
Jenkins	CI/CD pipeline for automated test execution
Jira	Test case management, defect tracking, sprint boards
Git / GitHub	Version control, branching, pull requests, code review
IntelliJ IDEA / Eclipse	Java IDE for automation framework development
Maven	Dependency management and build automation

12. Deliverables

Sprint	Deliverable
Sprint 1	Test Plan Document, Project Roadmap, Jira Board Setup, GitHub Repo
Sprint 2	Manual Test Cases (Jira), Initial Automation Framework, First Feature Files (BDD)
Sprint 3	Postman Collections + API Results, JMeter Scripts, SQL Query Scripts, Expanded POM Automation
Sprint 4	Optimized Framework, Full Execution Report, Newman Report, JMeter Report, Jenkins Pipeline, Test Completion Report, Framework User Guide

13. Risks and Mitigation

Risk	Likelihood	Impact	Mitigation
Demo website downtime / resets	Medium	High	Retry after delay; document results before reset; use local backup data
Dynamic locators breaking automation	Medium	High	Use stable locators (ID, data-*); add fallback XPath strategy
API endpoint changes on demo site	Low	Medium	Version-pin collections; re-validate before sprint execution
OpenCart DB access limitations	Medium	Medium	Use available read permissions; document any write restrictions
Team member availability gap	Low	Medium	Cross-training; assign backup per module; daily standups
Jenkins config complexity	Medium	Low	Use Jenkinsfile in repo; document setup steps in README

Inconsistent test data (demo resets)	High	Medium	Re-create test data before each run; use data factory in automation

14. Test Schedule

Activity	Start Date	End Date	Sprint
Test Plan Preparation	Week 1 Day 1	Week 1 Day 5	Sprint 1
Project & Tool Setup (Jira, GitHub, IDE)	Week 1 Day 3	Week 2 Day 2	Sprint 1
Exploratory Testing	Week 2 Day 1	Week 2 Day 3	Sprint 2
Manual Test Case Design	Week 2 Day 2	Week 3 Day 2	Sprint 2
API Test Case Design (Postman)	Week 2 Day 3	Week 3 Day 1	Sprint 2
DB Test Case Design (MySQL)	Week 2 Day 4	Week 3 Day 2	Sprint 2
Initial Automation Framework + BDD	Week 3 Day 1	Week 3 Day 5	Sprint 2
Full Tool Implementation (Newman, JMeter, MySQL)	Week 4 Day 1	Week 5 Day 2	Sprint 3
Full Test Execution (UI + API + DB + Perf)	Week 5 Day 3	Week 6 Day 3	Sprint 4
CI/CD Jenkins Integration	Week 6 Day 1	Week 6 Day 4	Sprint 4
Defect Analysis & Reporting	Week 6 Day 3	Week 7 Day 1	Sprint 4
Test Completion & Documentation	Week 7 Day 1	Week 7 Day 5	Sprint 4